



Exploring differences in self-regulated learning strategy use between high- and low-performing students in introductory programming: An analysis of eye-tracking and retrospective think-aloud data from program comprehension

Gary Cheng^{a,*}, Di Zou^b, Haoran Xie^c, Fu Lee Wang^d

^a Department of Mathematics and Information Technology, The Education University of Hong Kong, Hong Kong SAR, China

^b Centre for English and Additional Languages, Lingnan University, Hong Kong SAR, China

^c Department of Computing and Decision Sciences, Lingnan University, Hong Kong SAR, China

^d School of Science and Technology, Hong Kong Metropolitan University, Hong Kong SAR, China

ARTICLE INFO

Keywords:

Introductory programming
Self-regulated learning strategies
Eye tracking
Retrospective think aloud
Higher education

ABSTRACT

Previous studies have reported mixed results regarding the relationship between students' use of self-regulated learning (SRL) strategies and their performance in introductory programming courses. These studies were constrained by their reliance on self-report questionnaires as a means of collecting and analysing data. To address this limitation, this study aimed to employ eye-tracking and retrospective think-aloud techniques to identify differences in SRL strategy use for program comprehension tasks between high-performing students ($N = 31$) and low-performing students ($N = 31$) in an undergraduate programming course. All participants attended individual eye-tracking sessions to comprehend two Python program codes with different constructs. Their eye-tracking data and video-recalled retrospective think-aloud data were captured and recorded for analysis. The findings reveal that higher-order cognitive skills, such as elaboration and critical thinking, were mostly adopted by high-performing students, while basic cognitive and resource management strategy, such as rehearsal and help-seeking, were mostly employed by low-performing students when comprehending the program codes. This study not only demonstrates the design of combining eye-tracking and retrospective think-aloud data to explore students' use of SRL strategies but also provides evidence to support the notion that program comprehension is a complex process that cannot be effectively addressed by employing merely rudimentary strategies, such as repetitively reading the same code segment. In the future, researchers could explore the possibility of using a webcam to monitor and assess students' online programming processes and provide feedback based on their eye movements. They could also examine the effects of SRL strategies training on students' motivation, engagement, and performance in various types of programming activities.

* Corresponding author.

E-mail address: chengks@eduhk.hk (G. Cheng).

<https://doi.org/10.1016/j.compedu.2023.104948>

Received 30 April 2023; Received in revised form 25 August 2023; Accepted 13 October 2023

Available online 16 October 2023

0360-1315/© 2023 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY-NC license (<http://creativecommons.org/licenses/by-nc/4.0/>).

1. Introduction

Programming is commonly referred to as the process of designing, writing, testing and debugging a sequence of computer instructions designed to accomplish a task or solve a problem. Academic interest in the learning of programming could be traced back to the late 1960s when the Logo programming language was developed as a cognitive tool for children (Feurzeig et al., 1969). A potential benefit of programming is that it connects students with powerful ideas for problem solving through constructivism, where students are actively engaged in the development process of computer programs to construct their understanding of the world (Papert, 1980, 2000). In the past decade, there has been a growing and renewed emphasis on programming education due to its relationship with computational thinking (CT). CT has been widely recognised as an essential twenty-first century competence in our digital world (Grover & Pea, 2013), which ‘involves solving problems, designing systems, and understanding human behavior, by drawing on the concepts fundamental to computer science’ (Wing, 2006, p. 33). Programming can provide a means for all students, regardless of their education levels, to acquire and develop their CT (Lye & Koh, 2014; Romero et al., 2017).

Despite this, programming remains a challenging area of the computing curriculum for novices in higher education. Previous research has shown the difficulties novices face in formulating a programming problem, expressing a solution to the problem, executing, and evaluating the solution (Medeiros et al., 2019). This is particularly exemplified by the high dropout and failure rates in the introductory programming course at the university level, with approximately one-third of enrolled students unable to pass the course globally (Bennedsen & Caspersen, 2007; 2019; Watson & Li, 2014). To promote the programming skills of students, some studies have suggested that the contextual teaching factors such as curriculum content (Mehmood et al., 2020), choice of programming language (Koulouri et al., 2014), teaching methodology (Yan & Cheng, 2017; Malik & Coldwell-Neilson, 2017), assessment design (Veerasamy et al., 2019), and learning tools (Keuning et al., 2018) are important and should be carefully considered. Other studies have suggested that possible factors contributing to the understanding of programming concepts are the internal characteristics of students, such as learning style and self-efficacy (Cakiroglu, 2014; Shaw, 2012; Tsai, 2019).

However, research has shown that the contextual teaching factors were not significantly associated with student performance in the introductory programming course and has argued that the programming behaviours of individual students tended to play a more critical role (Pereira et al., 2020; Watson & Li, 2014). While exploring the causal attributions of students towards achievement outcomes in programming, Hawi (2010) found that the most often cited factors were learning strategy, followed by study habit and then by practice. These factors were considered relevant to the cognitive, metacognitive and resource management components of self-regulated learning (SRL), which were shown to be helpful for student learning in different disciplines (e.g. Akyol et al., 2010; Cheng & Chau, 2013; Zhang et al., 2020; Zheng et al., 2023). Surprisingly, Bergin et al. (2005) found that cognitive strategies (e.g. rehearsal, elaboration and organisation) did not appear to relate significantly to programming performance and indicated a need for further research to examine students’ strategy use in programming. Echeverry et al. (2018) found no correlation between programming performance and strategy use for one cohort of students but significant correlations between programming performance and the use of cognitive strategies, metacognitive strategies and resource management strategies for another cohort of students. Again, they suggested that further analysis should be undertaken to uncover the reasons for the correlation results.

An important limitation of previous studies assessing strategy use in introductory programming is that most of them relied exclusively on using a self-report questionnaire for data collection (e.g. Bergin et al., 2005; Echeverry et al., 2018; Song et al., 2021). There is a dearth of empirical studies providing a detailed analysis using a mixed methods approach. Additionally, limited efforts have been made to explore the differences between high- and low-performing students in terms of their strategy use for comprehension tasks with different programming constructs. To address these limitations, this study was designed to explore students’ use of SRL strategies for understanding program code through eye tracking and retrospective think aloud. The following two research questions (RQs) were specifically formulated in this study:

RQ1. How could eye-tracking and retrospective think-aloud data be combined for analysis to further our understanding of SRL strategy use for program comprehension?

RQ2. What are the differences between high- and low-performing students in terms of their SRL strategy use for program comprehension?

The findings of this study would contribute to a deeper, more data-driven understanding of the SRL strategies that are more closely linked to students’ learning performance in introductory programming. This knowledge can inform further research and practice in equipping novice students with effective SRL strategies to enhance their understanding of programming knowledge and skills.

2. Related work

2.1. Program comprehension

Program comprehension involves reading and understanding a given program code, which constitutes a crucial aspect of teaching and learning programming. It can often be described as a process in which individuals construct their mental model of a program’s data flow and functionality. Previous studies have proposed various models in an attempt to uncover the cognitive strategies used by programmers when comprehending a program, such as top-down, bottom-up, and integrated processing approaches (e.g. Brooks, 1983; Pennington, 1987; von Mayrhauser et al., 1997).

A top-down comprehension suggests that a programmer begins by drawing on his or her previous experience and domain

knowledge to formulate hypotheses about the main goals of the program. These hypotheses can subsequently be validated and refined based on the cues in the code, such as variable names or code patterns (Brooks, 1983). During the comprehension process, a programmer might use programming plans (i.e. stereotypic action sequences) and rules of programming discourse (i.e. coding conventions) to decompose goals into sub-goals (Soloway & Ehrlich, 1984). A bottom-up comprehension is the counterpart to the top-down approach, where a programmer seeks to first understand individual lines or sections of code and then integrates them into higher-level abstractions to infer the semantic meaning of the code (Shneiderman & Mayer, 1979; Pennington, 1987). An integrated approach builds on the previous conceptions of program comprehension by presenting a metamodel where programmers frequently shift between top-down (domain) and bottom-up (control and data flows) models to understand the program's purpose (von Mayrhauser & Vans, 1995). Shaft and Vessey (1995) argued that programmers would employ more top-down comprehension processes if they are well-versed in the application domain of the program; otherwise, they would rely more on bottom-up comprehension processes. O'Brien et al. (2004) demonstrated that expert programmers tend to use an integrated approach and that a higher level of familiarity with the application domain could lead to increased utilisation of the top-down, expectation-based comprehension process.

Schulte et al. (2010) identified that different models of program comprehension share three common elements: external representation, cognitive structure, and assimilation process. External representation pertains to the ways in which a program is presented. It exists outside an individual's internal knowledge, typically encompassing source code and other forms like documentation and diagrams. Cognitive structure refers to an individual's knowledge base concerning programming and application domains, coupled with his or her internal and cognitive representation of a program linked to the existing knowledge base. Assimilation process is concerned with the strategies an individual employs to construct his or her mental representation of a program from the external representation and existing knowledge base. Based on these commonalities across different models of program comprehension, Schulte et al. (2010) reviewed and characterised previous research, calling for further studies involving novices exposed to various forms of external representation. They also advocated the use of multiple methods, such as eye tracking and retrospective thinking aloud, to gather relevant data for analysis.

2.2. Eye tracking and program comprehension

Eye-tracking technology provides researchers with a window to measure and record the movement and fixation of human's eyes, enabling the investigation of visual attention, gazing patterns, and cognitive processes. By analysing where individuals are looking within a specific scene, how their gaze moves through the scene, and how long they spend looking at different regions of the scene (Holmqvist et al., 2011; Obaidellah et al., 2019), this technology has been widely applied to various research fields, including human-computer interaction (Conley et al., 2020), psychology (Rahal & Fiedler, 2019), neuroscience (Hessels & Hooge, 2019), and marketing (Boerman & Muller, 2022). It has also been employed in the field of program comprehension to study how programmers read, navigate, and comprehend program code for a better understanding of their program comprehension strategies (e.g. Jessup et al., 2021; Sharif et al., 2012).

One line of research into program comprehension with eye-tracking technology has been to examine differences in fixation data between novice and experienced programmers. Crosby and Stelovsky (1990) studied the eye movements of 19 novice and experienced programmers reading the English text and Pascal code of a binary search algorithm. They found that all programmers needed a higher number of fixations and regressions (jumps back) on the program code than the English text, suggesting that comprehending program code might be more cognitively demanding than natural language text. They also found that novice programmers fixated more on comments and comparisons in the program code, while experienced programmers fixated more on complex statements. Jessup et al. (2021) conducted a similar study to explore differences in program comprehension abilities between 10 novices and 10 experts who were asked to read and understand two pieces of program code (functioning as a default properties parser and an encryptor). They reported that experts had more fixation counts on program code than novices, but there was no significant difference in average fixation duration between novices and experts. The higher fixation counts of experts may partially be a feature of their engagement in a more complex process of program comprehension.

Research has also been directed towards using eye-tracking technology to identify differences in program comprehension strategies between novices and experts. Bednarik and Tukiainen (2006) compared the strategies of 16 novice and experienced programmers to comprehend short Java programs on recursive binary search and naïve string matching. A program visualisation tool was presented alongside the program code to enable the animation of program execution. Based on the analysis of attention allocation and fixation duration, it was found that experienced programmers tended to read the program code first, create their mental model and hypotheses about the program, and then confirm their hypotheses by running the animation at most twice. In contrast, novice programmers appeared to start by running the animation multiple times to gain a general understanding of the program and then focusing on the code representation. Busjahn et al. (2015) developed gaze-based measures (e.g. story order and execution order) to characterise the linearity of code reading order. They invited 14 novices and 6 experts to participate in reading a set of programs written in Java and in English pseudocode, followed by analysing their gaze data collected from an eye tracker during code reading. Their results indicated that all participants read program code less linearly than natural language text, and experts appeared to read program code less linearly than novices due to their advanced knowledge of program execution.

Regarding program comprehension for code debugging, Bednarik and Tukiainen (2008) analysed the eye-tracking recordings of 14 novice and experienced programmers who were involved in reading and debugging a program with non-syntactic errors. Three representations, including program code, program visualisation, and program output were presented in a debugging task. Experienced programmers were found to switch their visual attention more frequently between these three representations and to pay more attention to the program output at a later stage of debugging. In contrast, novice programmers did not often switch between the

representations and did not pay much attention to the program output. These findings suggest that experienced programmers are more likely to put effort into integrating information from different representations and linking program code to program output. The use of these strategies toward the end of debugging was found to be significantly correlated with debugging performance. [Uwano et al. \(2006\)](#) examined the eye movements of five graduate students with several years of programming experience to characterise their performance in identifying logical errors in six small-scale programs in C language. They found that most participants used the same strategy of first reading the whole program code from top to bottom (called the scan pattern), followed by focusing on certain parts of the code. They also found that the participants tended to identify errors efficiently if they spent sufficient time on scanning the entire code. [Sharif et al. \(2012\)](#) replicated the study by [Uwano et al. \(2006\)](#) with 15 participants and added that the scan time directly impacts the time taken and the visual effort required to detect code errors. They also found that experts tended to spend less time with the initial scan than novices because the former could rely on their internalised knowledge of programming plans to more efficiently spot certain lines of code with potential errors.

2.3. SRL strategies and programming performance

Self-regulated learning (SRL) can be defined as a constructive process in which learners actively set learning goals, monitor and evaluate their learning progress, and regulate and control their cognition, motivation, and behaviour to best accomplish their learning goals ([Pintrich, 2000](#)). It can also be described as a cyclical process that involves cognitive, metacognitive and motivational aspects occurring in the forethought, performance and self-reflection phases ([Zimmerman, 2000](#)). The cognitive and metacognitive aspects of SRL are particularly perceived as the 'skill' component comprising cognitive strategies, metacognitive strategies, and resource management strategies to support students in regulating their own learning ([Pintrich, 2004](#); [VanderStoep & Pintrich, 2008](#)). Cognitive strategies include rehearsal, elaboration, organisation and critical thinking ([Pintrich et al., 1993](#)). These strategies can help learners improve their working memory by reciting the information to be learned, linking new information to their existing knowledge, creating mental structures of learning materials for deeper understanding and longer retention, and applying their prior knowledge to new situations for problem solving or decision making. Metacognitive strategies refer to the activities of planning (e.g. setting goals for learning), monitoring (e.g. tracking progress made towards achieving the goals), and regulation (e.g. adjusting learning strategies for performance improvement), while resource management strategies primarily concern managing and controlling resources for learning such as the study environment, time, effort, and external assistance ([Pintrich, 1999](#)). In digital learning, cognitive strategies tend to be associated more with knowledge acquisition ([Greene et al., 2014](#)), while metacognitive strategies tend to contribute more to the understanding of the acquired knowledge ([Greene et al., 2018](#)).

Previous research has shown that students may adopt different SRL strategies to achieve success in different programming stages and situations. [Falkner et al. \(2014\)](#) analysed the self-reflections of 85 first-year university students from an introductory software development course to identify their use of SRL strategies leading to successful programming. Four key categories of programming-specific SRL strategies were identified, including design before coding, problem decomposition, time management and planning, and assessing task difficulty. [Pedrosa et al. \(2016\)](#) explored undergraduates' use of SRL strategy in a problem-based written assignment from their third programming course through thematic analysis of 401 weekly reflective entries over a ten-week period. They found that planning and organising strategies were most commonly adopted by students for information and knowledge search at an early stage of their assignment, while transformation strategies were more frequently used to apply previously acquired knowledge for problem solving afterwards. [Cakiroglu and Ozturk \(2017\)](#) identified the SRL strategy use of 30 undergraduates with problem-based learning in the flipped classroom through observation forms, interviews, and logs from an online learning platform. Their findings indicated that task strategies, monitoring strategies, and help-seeking strategies were adopted by more students in face-to-face sessions, while environment-structuring strategies were used by more students in home sessions.

In addition, previous research has sought to explore the differences between high- and low-performing students in their use of general strategies for programming, yet the findings are not entirely consistent. For example, [Robillard et al. \(2004\)](#) noted that high-performing individuals typically engaged in a systematic investigation of the code by adhering to a general plan, performing focused searches, and maintaining records of their results. This practice was not as prevalent among the low-performing group. [McCartney et al. \(2007\)](#) identified that high-performing students often employed strategies linked to a deep learning approach, such as connecting to mathematical formalism, to actively pursue an understanding of underlying concepts. In contrast, low-performing students tended to utilise strategies aligned with a surface learning approach, focusing on rote memorisation and drill. Further research by [Begum et al. \(2018\)](#) delved into these differences, highlighting that low-performing students often encountered difficulties in integrating different parts of a program and were uncertain about their programming progress. High-performing students, on the other hand, were more likely to reuse code, check specifications while coding, and possess knowledge of simpler coding methods. These high-performing students also tended to employ more effective strategies when tackling advanced problems, such as planning, visualising the solution, and extracting key elements of the problem.

When examining the use of SRL strategies among programming students, the Motivated Strategies for Learning Questionnaire (MSLQ) was commonly utilised as an assessment tool in previous studies. [Bergin et al. \(2005\)](#) scrutinised the use of SRL strategies by 35 undergraduate students in programming. Their findings highlighted that high-performing students employed metacognitive strategies (including planning, monitoring, regulating, and critical thinking) and resource management strategies (such as time and study environment, effort regulation, and peer learning) more frequently. However, there was no significant difference in the application of cognitive strategies (e.g. rehearsal, elaboration, organisation) among the students. Similarly, [Yukselturk and Bulut \(2005\)](#) assessed SRL components among 38 students in two online programming courses. They found that high-performing students predominantly used cognitive and metacognitive strategies, though a significant difference was only evident in the employment of metacognitive strategies

in the second course.

In contrast, [Fitzgerald et al. \(2005\)](#) found no substantial variation in the adoption of SRL strategies between high- and low-performing groups. They pointed out that nearly all students engaged in the walkthrough strategy when studying program code, albeit with varying degrees of success. This suggests that while the selection of strategies is crucial, the programming performance of students also hinges on how well these particular strategies are implemented. Consequently, it is necessary to devise a more refined data collection approach to ensure that the capture of subtle differences in how students apply SRL strategies. [Echeverry et al. \(2018\)](#) also sought to discern the correlation between SRL attributes and academic performance. Drawing data from 86 undergraduates enrolled in a programming course offered twice during an academic year, their research yielded varied results. In the first semester, they identified no correlations between the deployed SRL strategy and programming performance. However, the second semester revealed significant correlations, especially in the use of cognitive, metacognitive and resource management strategies – namely critical thinking, learning progress monitoring, and time management – with student performance.

Given the mixed results on the effectiveness of SRL strategies on programming, further investigation is warranted to identify the relationship between SRL strategy use and programming performance, as well as to explore the possible reasons for the identified relationship by using a mixed methods approach like observation or interview ([Echeverry et al., 2018](#)). Moreover, the measurement of SRL strategy with respect to programming could be based not only on self-report questionnaire data but also on more real-time and evidence-based data captured through other approaches to validate research findings ([Arakawa et al., 2021](#)).

3. Method

3.1. Study context

This study is part of a larger project, namely “Tracking Students’ Use of Self-Regulated Learning Strategies for Computer Programming in Teacher Education”. It was conducted at a Hong Kong university with teacher education programmes between 2021 and 2022. The study aimed to investigate the use of SRL strategy by high- and low-performing undergraduate students for program comprehension, utilising eye-tracking and retrospective think-aloud data. Participants in this study were students enrolled in a 13-week, credit-bearing course titled ‘Introduction to Programming and Problem Solving’. The programming language used throughout the course was Python. Upon completing the course, students were expected to understand and develop program code to solve simple real-world problems.

3.2. Participants

A total of 62 undergraduates pursuing the Bachelor of Education in Primary Mathematics provided their written consent to participate in this study. Prior to the commencement of the study, all participants filled out a questionnaire providing their basic demographic information, such as gender, age, programming skills, and experience. Among the participants, 35 were males and 27 were females, with ages ranging from 19 to 24 years ($M = 21.2$, $SD = 1.5$). All participants reported in the questionnaire that they had no prior knowledge or experience in computer programming.

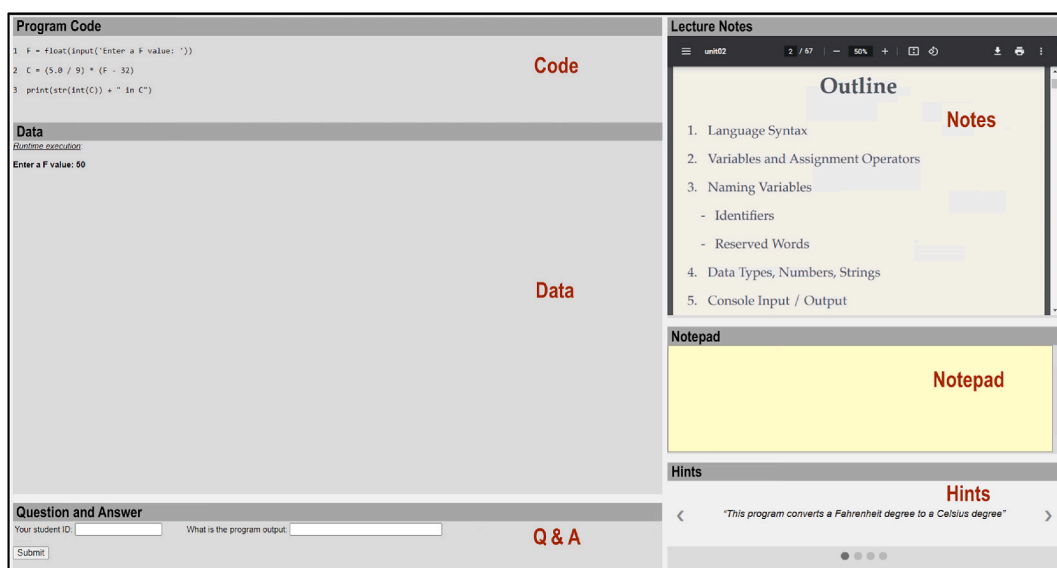


Fig. 1. The user interface used in the eye-tracking experiment.

3.3. Experimental setup

Each participant was invited to read and understand the source codes of two Python programs in an eye-tracking experiment. The experiment took place in a room equipped with a dedicated computer, a Gazepoint GP3 HD eye tracker with a sampling rate of 60 Hz, and Gazepoint Analysis UX Edition software. The participant's gaze behaviour in the experiment was recorded by the eye tracker and then analysed by the software on the computer. As shown in Fig. 1, a user interface was specifically designed to display different forms of external representation and support on the computer screen. It was divided into six regions, including: (1) *Code*, which displayed the source code of a Python program, (2) *Data*, which indicated the input data for the program at runtime, (3) *Q&A*, which asked a question about the program and collected answers, (4) *Notes*, which allowed participants to search for relevant information from lecture notes, (5) *Notepad*, which allowed participants to jot down any information during program comprehension, and (6) *Hints*, which explained the semantic meaning of the program.

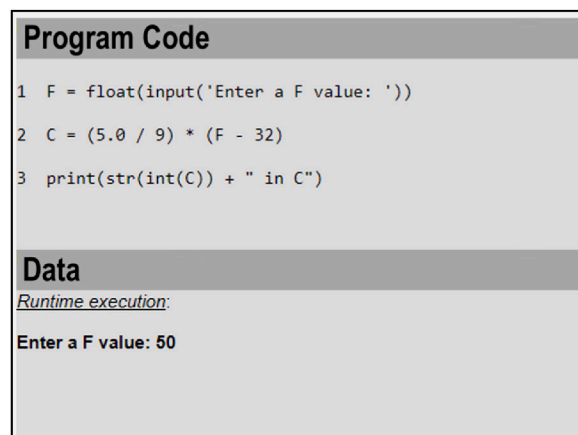
The experiment included two basic programs written in Python. The first program (referred to as 'Program Code 1') dealt with operators and expressions, requiring participants to perform computations on an input value and then generate an output string (see Fig. 2). The second program (referred to as 'Program Code 2') focused on control structures, where participants were tasked with repeating the execution of a conditioned code block and then determining the result (see Fig. 3). Semantically, the first program was designed to convert a temperature in Fahrenheit to Celsius, while the second program aimed to find the maximum value from a given list of numbers.

3.4. Measures

3.4.1. Exploration of SRL strategy use through eye tracking and retrospective think aloud

To monitor participants' gaze positions and eye movements during code reading, a research-grade Gazepoint GP3 HD eye tracker with a 60 Hz sampling rate was utilised. The eye tracker offers 0.5–1 degree of visual angle accuracy and provides movement range of 25 cm (horizontal) x 11 cm (vertical), with ± 15 cm depth movement range, and 5- and 9-point calibration. Previous research has validated the eye tracker's accuracy, precision and robustness in data collection (Brand et al., 2021; Cuve et al., 2021). In this study, all eye-tracking sessions were live-controlled and recorded using Gazepoint Analysis UX software.

Through the analysis of eye-tracking metrics on areas of interest (AOIs), the SRL strategies employed by participants during eye-tracking sessions were explored. Each region of the user interface on computer screen was designated as an independent AOI. The number of eye fixations on a specific AOI (commonly known as fixation count) and the total duration of fixations divided by the number of fixations within a specific AOI (referred to as average fixation duration) were measured to assess participants' visual attention and cognitive workload. To determine which regions of the user interface held the most salience for participants and how attention was distributed across these regions, transition matrices were employed to quantify the probability of eye gaze movements from one region to another. Furthermore, participants' eye movements (i.e. scanpaths), formed by saccades between fixations on AOIs, were extracted and analysed using Scanpath Trend Analysis (STA) to identify the most common representative order of attention points for each group (Eraslan et al., 2016). STA employs hierarchical clustering and applies the String-edit algorithm, without the need for parameter settings, to identify the two most similar scanpaths from a list of scanpaths obtained from a specific group of students. It then determines their longest common subsequence (LCS). By repeatedly removing two scanpaths and introducing their LCS back into the list, the process ultimately results in a single representative scanpath that encapsulates the general pattern followed by the student group. The technical details of the algorithm for STA are presented in Fig. 4. Lastly, to gain a more in-depth understanding of the SRL strategies used by the participants, a fusion of eye-tracking and retrospective data was adopted. A retrospective think-aloud method was implemented where participants were asked to watch playback of their own gaze videos. These videos displayed participants' fixation points and scanpaths overlaid on the user interface, recorded during their eye-tracking sessions. This



```

Program Code

1 F = float(input('Enter a F value: '))
2 C = (5.0 / 9) * (F - 32)
3 print(str(int(C)) + ' in C')

Data
Runtime execution:
Enter a F value: 50

```

Fig. 2. A Python program using operators and expressions (Program Code 1).

Program Code

```
1 n = int(input('Enter a number: '))
2 m = n
3 while n != 0:
4     n = int(input('Enter a number: '))
5     if n > m:
6         m = n
7 print(str("m is " + str(m) + ", n is " + str(n)))
```

Data

Runtime execution:

Suppose the following numbers are entered in sequence:

2

3

4

5

0

9

6

Fig. 3. A Python program using control structures (Program Code 2).

process enabled participants to recall and verbalise their thoughts and actions during the tasks, effectively connecting real-time eye-tracking data to retrospective verbal insights (Schulte et al., 2010).

3.4.2. Measurement of programming achievement by course assessment results

Participants' programming achievements were evaluated using their assessment results in the introductory programming course. The course assessment comprised three components: programming exercises, a mini programming project, and a written examination, collectively providing a comprehensive evaluation of students' theoretical knowledge and practical skills in programming. Programming exercises contributed to 30% of the overall grade and covered fundamental programming concepts such as variables, control structures, functions, and data structures. The mini programming project, accounting for 40%, tasked students with designing and implementing a small program to address a specified problem statement. The written examination, constituting 30% of the grade, assessed students' understanding of programming concepts and their ability to write accurate and efficient code. At the commencement of the course, participants were thoroughly informed about the assessment tasks and their respective weightings. These assessments were individually graded by the same experienced teacher.

3.5. Procedure

Eye-tracking experiments were performed during weeks 5–6 of the semester when participants had acquired programming concepts related to operators, expressions, and control structures. Prior to the experiments, participants completed a demographic questionnaire to provide background information. Subsequently, participants were individually invited to attend eye-tracking sessions to comprehend two Python program codes. At the beginning of each session, the eye tracker was calibrated for each participant using a

Preliminary Stage:

Step 1. Represent each individual scanpath as a sequence of AOIs with fixation durations

First Pass Stage:

Step 2. Identify trending visual element instances for all individual scanpaths

- (a) *Number every instance of non-consecutively repeated AOIs in each scanpath based on the total fixation duration on each instance, where the longest instance of a particular AOI is numbered by 1*
- (b) *Identify trending instances of AOIs which meet the following two requirements:*
 - *The total fixation count (TFC) on a trending instance must not be smaller than the minimum TFC on the instances fully shared by all individual scanpaths*
 - *The total fixation duration (TFD) on a trending instance must not be smaller than the minimum TFD on the instances fully shared by all individual scanpaths*

Second Pass Stage:

Step 3. Combine all trending instances to generate the trending path based on their positions

- (a) *Merge the consecutive repetitions of the same trending instance into one single instance with the fixation duration equivalent to the sum of the fixation durations of the same consecutive trending instances*
- (b) *Compute the individual priority value (IPV) of each trending instance in each scanpath with the following equation:*

$$IPV = 1 - P \times \frac{\max - \min}{L - 1}$$

where:

P is the instance position in the scanpath, starting from 0

L is the length of the user scanpath

max is the maximum priority value (default value is 1)

min is the minimum priority value (default value is 0.1)

- (c) *Compute the total priority value (TPV) of each trending instance with the following equation:*

$$TPV = \sum_{i=1}^n IPV_i$$

where:

IPV_i is the individual priority value of the ith scanpath

n is the total number of scanpaths

- (d) *The position of a trending instance in the trending path is determined by its total priority value, where the instance with the highest total priority value is positioned at the front*

Fig. 4. Algorithm for STA

Fig. 4. Algorithm for STA

9-point calibration method, and the calibration was performed twice to achieve higher accuracy. Participants were then instructed to read the program codes and respond to questions about the program outputs. An eye tracker captured and recorded their eye movements during the program comprehension process. There was no time limit for each eye-tracking session, and a session was stopped once the participant submitted his or her answers. Following each session, participants were asked to watch the playback of their eye movements and verbalise their thought process regarding their approach to answering the questions. On average, each experiment, comprising both eye tracking and retrospective think aloud, lasted 16 min.

At the end of the introductory programming course, the teacher assigned scores for course assessments to each student. This study adopted an ex-post-facto approach to examine the differences in SRL strategy use for program comprehension between high- and low-performing students. The objective was to investigate the impact of performance level on SRL strategy utilisation during program comprehension. SRL strategy use was identified as the independent variable in this ex-post-facto approach. This factor, already having occurred, is unlikely to change significantly over a short duration without direct and explicit instruction (Michalsky, 2021). Consequently, it is reasonable to expect that students' use of SRL strategies would remain relatively consistent throughout the study period. The study aimed to observe the influence of this independent variable on the students' performance level at the end of the course. Participants were categorised into two groups for comparison based on their performance scores. Those with scores higher than the median were classified as the high-performing group (HG), while the remaining participants were placed in the low-performing group

(LG). A median split of participants' performance scores resulted in 31 participants being assigned into each group. Descriptive statistics for the overall programming performance scores of both groups are presented in Table 1. All statistical analyses in this study were conducted using the Statistical Package for the Social Sciences (SPSS), version 28 for Mac.

4. Results

4.1. Fixation measurements

The fixation count and average fixation duration (in milliseconds) on each AOI of the user interface for Program Code 1 and Program Code 2 were measured and compared between LG and HG. Box plots were employed to illustrate the overall distribution of fixation count (see Fig. 5) and average fixation duration (see Fig. 6) on each AOI across the two program codes, showcasing their variability within the two groups. These plots offer a visual means to compare the fixation measurements between the groups through their overall distributions. Data normality was assessed using the Shapiro-Wilk tests, and the results demonstrated strong evidence of non-normality. Due to the violation of the assumption of data normality, non-parametric Mann-Whitney U tests were conducted to ascertain significant differences in the fixation measurements between the groups. Separate analyses were performed for Program Code 1 and Program Code 2 due to variations in the technical features and text length of the two programs.

Table 2 shows the comparison results between LG and HG for Program Code 1. Results from Mann-Whitney U tests reveal significant differences between LG and HG in fixation counts on five AOIs: Code ($U = 10.500$, $|Z| = 6.618$, $p < 0.001$), Data ($U = 173.000$, $|Z| = 4.333$, $p < 0.001$), Notes ($U = 75.000$, $|Z| = 5.713$, $p < 0.001$), Notepad ($U = 170.000$, $|Z| = 4.734$, $p < 0.001$) and Hints ($U = 242.000$, $|Z| = 3.361$, $p < 0.001$). Students' programming performance is shown to have a large effect on their fixation counts on Code ($r = 0.840$), Notes ($r = 0.726$), Notepad ($r = 0.555$) and Data ($r = 0.550$), along with a near-large effect on their fixation count on Hints ($r = 0.427$) based on Cohen's criteria for small ($r = 0.10$), medium ($r = 0.30$) and large ($r = 0.50$) effects (Cohen, 1988). Additionally, the results indicate that the two groups did not differ significantly in terms of the number of fixations on Q&A and the average fixation durations on all AOIs.

Table 3 displays the comparison results between LG and HG for Program Code 2. Results from Mann-Whitney U tests show a significant group difference in fixation counts, specifically on Notepad ($U = 200.500$, $|Z| = 3.946$, $p < 0.001$) and Hints ($U = 227.500$, $|Z| = 3.574$, $p < 0.001$). These results also indicate a large effect of programming performance on fixation counts on Notepad ($r = 0.501$) and a near-large effect of programming performance on fixation counts on Hints ($r = 0.427$). However, no statistically significant differences between the two groups were observed in fixation counts on the four remaining AOIs (Code, Data, Q&A, Notes), nor in average fixation durations on all AOIs.

4.2. Visual transitions and trending scanpaths

The analysis of visual transitions was conducted using transition matrices designed to visualise participants' gaze transitions between AOIs. Fig. 7 depicts the transition matrices, showcasing the normalised probability of LG and HG shifting their gaze from one AOI to the next in both Program Code 1 and Program Code 2. Darker shades within the matrices represent higher probabilities of gaze transitions from a specific preceding AOI to the current AOI.

In Fig. 7, three distinct patterns emerged between LG and HG. Firstly, LG demonstrated more frequent gaze transitions from Code to Notes in Program Code 1 and from Code to Hints in Program Code 2. In contrast, HG consistently exhibited greater gaze transitions from Code to Data for both Program Code 1 and Program Code 2. Secondly, when focusing on the aids, LG demonstrated a tendency to direct more gaze transitions towards Notes (particularly pronounced in Program Code 1) and Hints (notably in Program Code 2) compared to Notepad. In contrast, HG showed an opposite gaze pattern, allocating a higher proportion of visual attention to Notepad in Program Code 2 than in Program Code 1. Lastly, LG's gaze transitions showed a greater shift from Hints to Data, while HG's transitions were more pronounced from Hints to Notepad.

Furthermore, Scanpath Trend Analysis (STA) was implemented to identify the trending scanpath for individuals within each group. The trending scanpath signifies the most frequently observed eye movement pattern across multiple participants (Eraslan et al., 2016). Fig. 8 visually represents the scan patterns of LG and HG from their respective trending scanpaths for Program Code 1 and Program Code 2. The sequence of AOIs visited by participant groups (y-axis) was directly extracted from their trending scanpath, while the time spent on each AOI (x-axis) was determined by its median fixation duration across the participants. To refine the analysis at a more finer-grained level, each line of code in the Code area was treated as an AOI, denoted by Code-Lx, where x represents the line number of the code.

As shown in Fig. 8, LG and HG demonstrated notable differences in their eye movements during code comprehension. For Program

Table 1
Descriptive statistics for the programming performance scores of LG and HG.

Group	No. of Participants	Overall Programming Performance Score		
		Range	Mean	S.D.
LG	31	46.9–72.0	63.0	6.2
HG	31	72.3–87.6	78.3	4.4

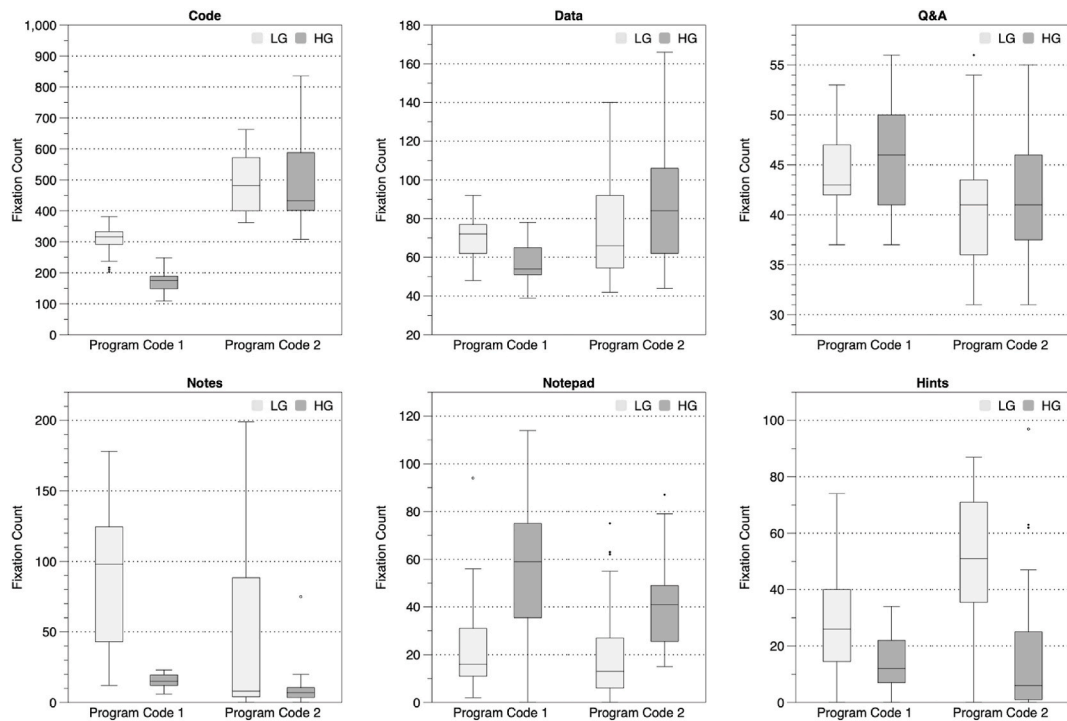


Fig. 5. Fixation counts of LG and HG on different AOIs.

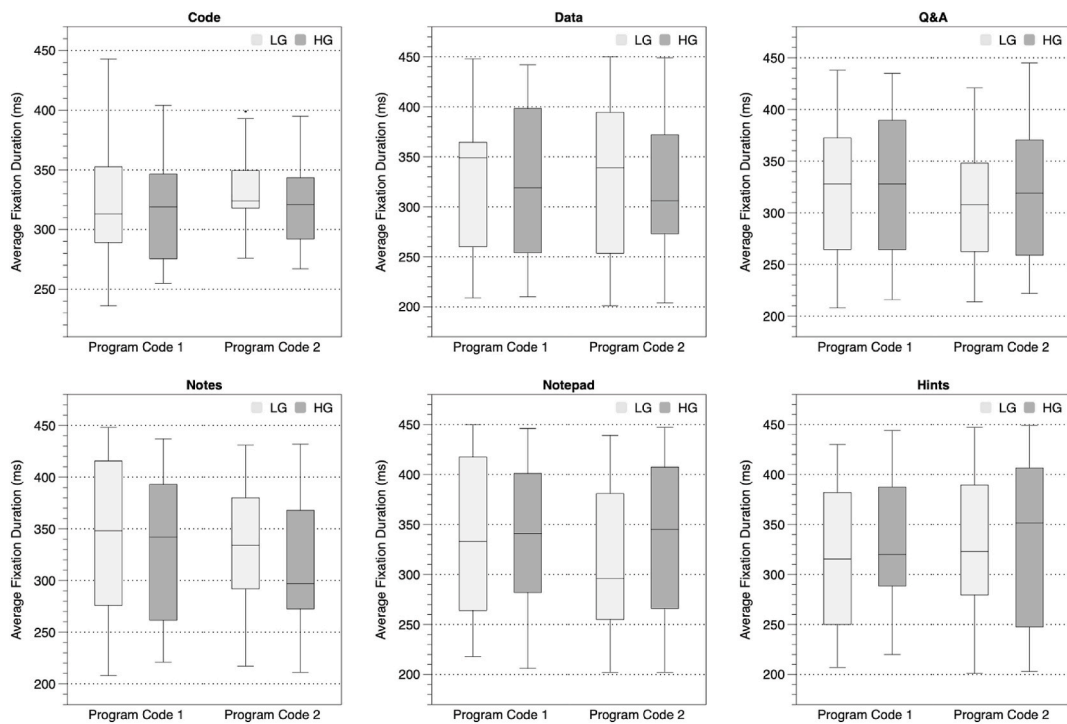


Fig. 6. Average fixation durations of LG and HG on AOIs.

Code 1, LG displayed a tendency to engage with the code in a non-linear manner and initiate fixations on Notes at an early stage. They also demonstrated frequent attention switches between Code and Notes for Program Code 1, and between Code and Hints for Program Code 2, with relatively limited focus on Notepad. The time spent on comprehending the two program codes was comparable. Conversely, HG

Table 2

Comparison of visual attention to AOIs between LG and HG for Program Code 1.

AOIs	Measures	Mean Rank		U	Z	Asymp. Sig. (2-tailed)	Effect Size <i>r</i>
		LG (N = 31)	HG (N = 31)				
Code	FC	46.66	16.34	10.500	−6.618	0.000***	0.840
	AFD	31.92	31.08	467.500	−0.183	0.855	0.023
Data	FC	41.42	21.58	173.000	−4.333	0.000***	0.550
	AFD	31.03	31.97	466.000	−0.204	0.838	0.026
Q&A	FC	29.21	33.79	409.500	−1.002	0.316	0.127
	AFD	31.21	31.79	471.500	−0.127	0.899	0.016
Notes	FC	44.58	18.42	75.000	−5.713	0.000***	0.726
	AFD	33.42	29.58	421.000	−0.838	0.402	0.106
Notepad	FC	21.48	41.52	170.000	−4.374	0.000***	0.555
	AFD	31.74	31.26	473.000	−0.106	0.916	0.013
Hints	FC	39.19	23.81	242.000	−3.361	0.000***	0.427
	AFD	30.81	32.19	459.000	−0.303	0.762	0.038

****p* < 0.001.**Table 3**

Comparison of visual attention to AOIs between HG and LG for Program Code 2.

AOIs	Measures	Mean Rank		U	Z	Asymp. Sig. (2-tailed)	Effect Size <i>r</i>
		LG (N = 31)	HG (N = 31)				
Code	FC	32.81	30.19	440.000	−0.570	0.569	0.072
	AFD	35.19	27.81	366.000	−1.612	0.107	0.204
Data	FC	27.95	35.05	370.500	−1.550	0.121	0.197
	AFD	32.45	30.55	451.000	−0.415	0.678	0.053
Q&A	FC	29.65	33.35	423.000	−0.812	0.417	0.103
	AFD	30.52	32.48	450.000	−0.429	0.668	0.054
Notes	FC	35.63	27.37	352.500	−1.806	0.071	0.229
	AFD	34.35	28.65	392.000	−1.247	0.213	0.158
Notepad	FC	22.47	40.53	200.500	−3.946	0.000***	0.501
	AFD	26.18	36.82	315.500	−2.324	0.020*	0.295
Hints	FC	39.66	23.34	227.500	−3.574	0.000***	0.454
	AFD	32.38	30.61	453.000	−0.388	0.698	0.049

****p* < 0.001; **p* < 0.05.

appeared to follow a more sequential approach when reading the code. They initially directed their attention between Code and Data, subsequently allocating more attention to Notepad at a later stage, albeit with limited attention to both Notes and Hints. The time spent on comprehending the second program code exceeded that of the first one.

4.3. Video-recalled retrospective think aloud

A qualitative analysis was performed on the video-recalled retrospective think-aloud data to identify the presence of SRL strategies. Initially, a set of codes for SRL strategies specific to programming were developed according to the seminal work on SRL strategies scales (Pintrich et al., 1991). Drawing on the open coding scheme in grounded theory (Charmaz, 2014), codes were added and refined as new SRL strategies emerged from the think-aloud data. The coding process was performed by two researchers from the project team. They independently analysed the data, identifying the strategies adopted by participants and the scanpath patterns that signified those strategies. The analysis was based on the thoughts verbalised by participants, as audio-recorded in a synchronous way while they watched replays of their eye-tracking videos. For every action verbally articulated by a participant within a specific time frame, the corresponding scanpath was extracted and examined for that exact duration. Repetitive sub-sequences of visual elements in a scanpath were generalised and eliminated to retain the most representative pattern for a particular action. A strategy type and a code were then assigned to each new pair of pattern and action that emerged from the data. An illustrative example of this process might involve a participant reading program code and jotting down their interpretation in Notepad. In this scenario, the scanpath might progress as follows: L1 → L2 → Notepad → L2 → Notepad → L3 → Notepad. This sequence could initially be generalised to Code → Notepad → Code → Notepad → Code → Notepad and ultimately distilled to a representative pattern of Code → Notepad. After completing the coding process, coding discrepancies were discussed and coding definitions were adjusted until consensus was reached to ensure consistency. Cohen's kappa inter-coder reliability score was computed to be 97.5%, demonstrating nearly perfect agreement of coding between coders (Landis & Koch, 1977).

Table 4 outlines the SRL strategies with their corresponding scanpath patterns, as identified from the coding process. The SRL strategies fall into two categories: cognitive and resource management strategies. Cognitive strategies, including rehearsal, elaboration, organisation, and critical thinking, aim at processing and making sense of information (Pintrich et al., 1993). Rehearsal involves

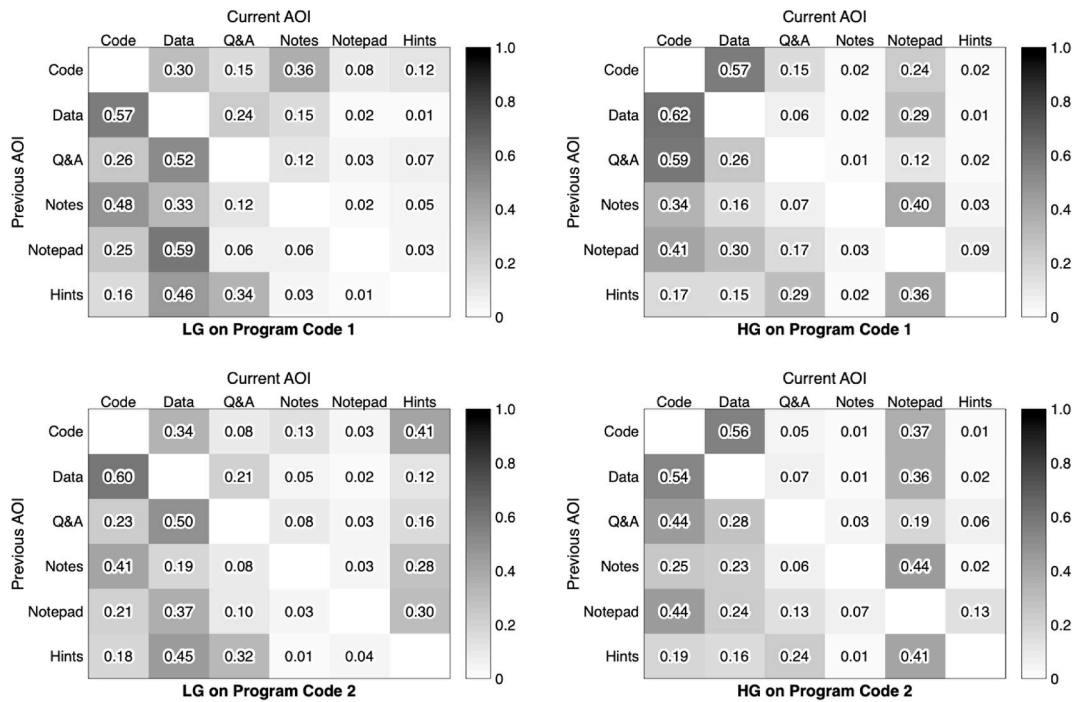


Fig. 7. The transition matrices of LG's and HG's gaze.

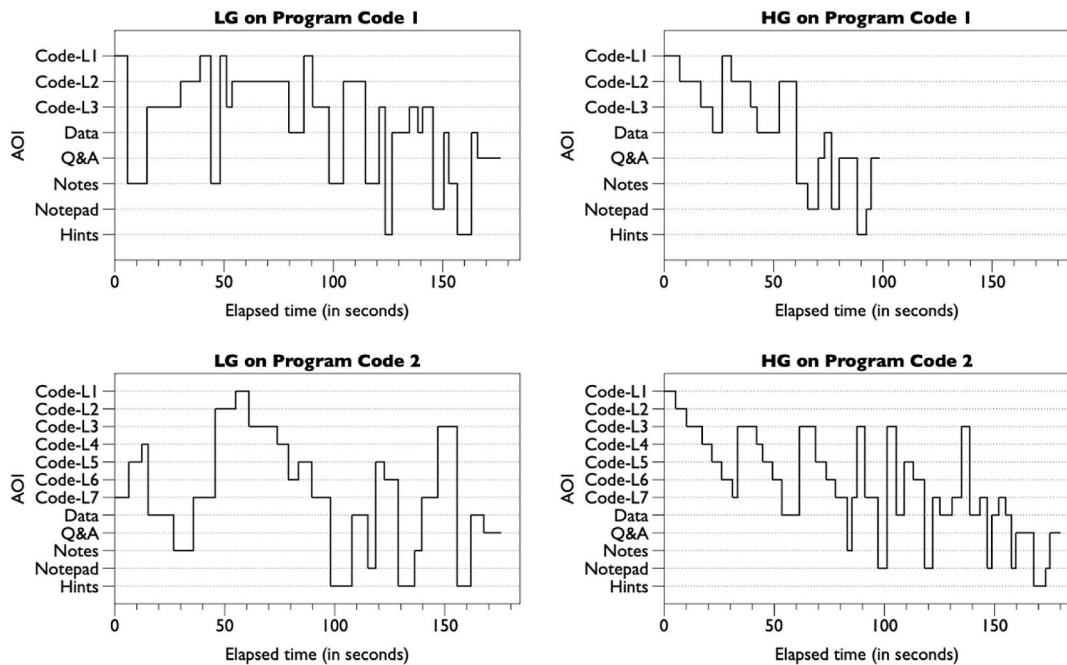


Fig. 8. Visualisation of LG's and HG's scan patterns from their trending scanpaths.

repetitively going over information to strengthen memory and recall, without necessarily reflecting deep processing. Elaboration, through summarising and note-taking, facilitates the integration of new information with prior knowledge. Organisation, like structuring and grouping, supports learners in selecting relevant information, linking it together, and putting it into a logical format. Critical thinking involves applying previous knowledge to new situations for problem-solving, decision-making, or evaluation (Cheng, Poon, Lau, & Zhou, 2019). On the other hand, resource management strategies, such as effort regulation and help seeking, are concerned

Table 4

List of SRL strategies with corresponding scanpath patterns.

Strategy Type	Strategy	General Description	Scanpath		
			Code	Pattern	Action
Cognitive strategy	Rehearsal	Repetition of information to aid in its encoding and retention in working memory	R	$\{\text{Code}_x\} \rightarrow \{\text{Code}_y\}$ where $\{\text{Code}_x\}$ and $\{\text{Code}_y\}$ denote two sets of code lines and $\{\text{Code}_x\} \geq \{\text{Code}_y\}$	Repeated reading the same instructions
	Elaboration	Establishment of internal connections between information to be processed and understood	E1	Code $\rightarrow$ Notepad	Recording or recalling observations about the code
			E2	Notes $\rightarrow$ Notepad	Summarising or retrieving key points from the notes
			E3	Data $\rightarrow$ Notepad $\rightarrow$ Code Code $\rightarrow$ Notepad $\rightarrow$ Data Notepad $\rightarrow$ Code $\rightarrow$ Data	Jotting down or referring to intermediate results
			E4	Code $\rightarrow$ Data $\rightarrow$ Code	Passing data to code
	Organisation	Selection of appropriate information and constructing connections among information	O	Notepad $\rightarrow$ Q&A	Structuring the answer in a suitable format
	Critical thinking	Analysis and evaluation of information to form a judgement	CT1	Q&A $\rightarrow$ Hints	Verifying the answer with hints
			CT2	Hints $\rightarrow$ Notepad $\rightarrow$ Q&A	Cross-checking the intermediate steps against hints to confirm the answer
			CT3	Hints $\rightarrow$ Data $\rightarrow$ Q&A	Estimating the answer by linking hints with data
Resource management strategy	Effort regulation	Regulation of effort and attention towards problem solving	ER	Notes $\rightarrow$ Hints	Trying an alternative way to get assistance
	Help seeking	Help seeking from external sources	HS	Code $\rightarrow$ Notes $\rightarrow$ Code Code $\rightarrow$ Hints	Gaining a better understanding of code semantics from extra information

with controlling and managing additional resources to improve comprehension (Pintrich, 1999). Effort regulation refers to the monitoring and control of one's efforts and attention to complete tasks, while help seeking engages learners in actively seeking assistance from others when faced with challenges (Bergin et al., 2005).

The normality of the data regarding strategy use was assessed using Shapiro-Wilk tests, which revealed that the data did not follow a normal distribution. As a result, non-parametric Mann-Whitney U tests were employed to determine whether there were significant differences in strategy use between LG and HG. The results for Program Code 1 and Program Code 2 are presented in Table 5 and Table 6, respectively.

Based on Mann-Whitney U tests, the results for Program Code 1 show statistically significant differences between LG and HG in the use of rehearsal strategy ($U = 274.000$, $|Z| = 2.913$, $p < 0.01$), elaboration strategy E3 ($U = 311.000$, $|Z| = 2.459$, $p < 0.05$), organisation strategy ($U = 340.000$, $|Z| = 2.287$, $p < 0.05$), critical thinking strategy CT2 ($U = 240.000$, $|Z| = 3.714$, $p < 0.001$) and help seeking ($U = 212.000$, $|Z| = 3.824$, $p < 0.001$), with effect sizes ranging from near-medium ($r = 0.290$) to near-large ($r = 0.486$). There was no statistically significant difference in the utilisation of effort regulation strategy between the two groups.

The results for Program Code 2 also show statistically significant differences between LG and HG in the use of rehearsal strategy ($U = 302.000$, $|Z| = 2.517$, $p < 0.05$), elaboration strategies E1 to E4 ($U = 264.000$ – 321.000 , $|Z| = 2.355$ – 3.218 , $p < 0.05$), critical thinking strategies CT1 to CT3 ($U = 225.000$ – 946.000 , $|Z| = 2.702$ – 6.590 , $p < 0.01$) and help seeking ($U = 144.000$, $|Z| = 4.822$, $p < 0.001$), with effect sizes ranging from near-medium ($r = 0.299$) to large ($r = 0.837$). However, unlike the results for Program Code 1, there was statistically significant difference in the utilisation of effort regulation strategy between the two groups ($U = 247.000$, $|Z| =$

Table 5

Comparison of strategy use between LG and HG for Program Code 1.

Strategy	Scanpath code	Mean Rank		U	Z	Asymp. Sig. (2-tailed)	Effect Size r
		LG (N = 31)	HG (N = 31)				
Rehearsal	R	38.16	24.84	274.000	−2.913	0.004**	0.370
Elaboration	E1	28.56	34.44	389.500	−1.312	0.190	0.167
	E2	28.06	34.94	374.000	−1.536	0.124	0.195
	E3	26.03	36.97	311.000	−2.459	0.014*	0.312
	E4	34.39	28.61	391.000	−1.263	0.207	0.160
Organisation	O	26.97	36.03	340.000	−2.287	0.022*	0.290
Critical thinking	CT1	34.69	28.31	381.500	−1.583	0.113	0.201
	CT2	23.74	39.26	240.000	−3.714	0.000***	0.472
	CT3	34.42	28.58	390.000	−1.561	0.119	0.198
Effort regulation	ER	34.81	28.19	378.000	−1.921	0.055	0.244
Help seeking	HS	40.16	22.84	212.000	−3.824	0.000***	0.486

*** $p < 0.001$; ** $p < 0.01$; * $p < 0.05$.

Table 6

Comparison of strategy use between LG and HG for Program Code 2.

Strategy	Scanpath code	Mean Rank		<i>U</i>	<i>Z</i>	Asymp. Sig. (2-tailed)	Effect Size <i>r</i>
		LG (<i>N</i> = 31)	HG (<i>N</i> = 31)				
Rehearsal	R	37.26	25.74	302.000	−2.517	0.012*	0.320
Elaboration	E1	24.52	38.48	264.000	−3.218	0.001**	0.409
	E2	26.35	36.65	321.000	−2.355	0.019*	0.299
	E3	25.32	37.68	289.000	−3.039	0.002**	0.386
	E4	25.56	37.44	296.500	−2.592	0.010*	0.329
Organisation	O	31.68	31.32	475.000	−0.126	0.899	0.016
Critical thinking	CT1	23.26	39.74	225.000	−3.821	0.000***	0.485
	CT2	16.48	46.52	946.000	−6.590	0.000***	0.837
	CT3	36.03	26.97	340.000	−2.702	0.007**	0.343
Effort regulation	ER	39.03	23.97	247.000	−3.718	0.000***	0.472
Help seeking	HS	42.35	20.65	144.000	−4.822	0.000***	0.612

*** $p < 0.001$; ** $p < 0.01$; * $p < 0.05$.

3.718, $p < 0.001$), with a near-large effect size ($r = 0.472$). In addition, no statistically significant difference was found in the utilisation of organisation strategy between the two groups.

5. Discussion

This study was formulated to examine low- and high-performing students' (LG and HG) employment of SRL strategies in comprehending program code with different constructs, utilising eye-tracking and retrospective think-aloud techniques. The data on fixation measurements (Obaidellah et al., 2019), visual transitions and trending scanpaths (Eraslan et al., 2016), and video-recalled retrospective think aloud (Schulte et al., 2010) were gathered for fine-grained analysis to address the research questions.

Fixation measurements, including fixation count and average fixation duration, were collected from LG and HG on different AOIs within the user interface. This was done to compare their visual attention and cognitive workload. For Program Code 1 (without selection and repetition statements), results show that LG had significantly higher fixation counts on the code and data compared to HG, indicating that comprehending a simple sequential code was a task with a high cognitive workload for LG. They also show that LG demonstrated higher fixation counts on the notes and hints, potentially indicating an intention to rely on passively receiving information from available aids to determine the outputs of the program code. Such an approach, characterised by using external resources to directly obtain answers without investing substantial effort into understanding the code, was identified as a kind of surface learning approach and also a typical trait of low-performing student in previous research (McCartney et al., 2007). On the other hand, HG exhibited a significantly higher number of fixations on the notepad, suggesting that they relied on the support for actively processing and organising information. For Program Code 2 (with selection and repetition statements), there were no significant differences between LG and HG in fixation counts on the code and data. This finding suggests that comprehending the code with non-sequential control structures required a similarly high level of cognitive workload to both LG and HG. In order to understand a more challenging code and answer the question, LG tended to rely more on the hints and less on the notes, while HG tended to continue using the notepad, as evidenced by their fixation counts on the respective aids. These findings corroborate and substantiate prior observations that high-performing individuals typically approached program code in a systematic manner by extracting key information (Begum et al., 2018) and keeping records of intermediate results from the program code (Robillard et al., 2004). This approach was rarely observed among low-performing students.

For further analysis, visual transitions from one AOI to the next one were observed and recorded between LG and HG. The trending scanpaths of LG and HG were also generated to show their most common visual attention patterns for comparison. Results indicate that LG appeared to read program code in a non-linear manner compared to HG. They also indicate that LG showed a higher number of gaze transitions from the code to the notes when reading Program Code 1 (a simpler program), but more gaze transitions from the code to the hints when reading Program Code 2 (a more challenging program). On the other hand, HG consistently demonstrated more gaze transitions from the code to the data for both programs and displayed a greater proportion of visual attention towards the notepad in Program Code 2 compared to Program Code 1. The findings suggest that low-performing students may struggle with comprehending program code in a sequential and systematic manner, possibly indicating difficulties in understanding the logic and structure of the code. This could account for their tendency to access additional information, such as the notes and hints, while interpreting and executing the code.

LG tended to rely on the notes for a simpler programming task, as it provides clear explanations of the syntax, semantics, and logic of a programming language. However, the notes may assume a certain level of prior knowledge, which could only be beneficial for novices to comprehend simple programs. On the other hand, for a more challenging programming task, LG tended to rely on the hints, which may require less reliance on prior knowledge and provide more simplified explanations and explicit guidance in plain language. While this approach may make it easier for novices to understand complex programs, it might hinder their ability to critically connect the program code, input and output. As Bednarik and Tukiainen (2008) suggested, limited attentional switches between program code and its associated parts (e.g. input and output) could adversely affect programming performance. In contrast, HG typically followed a more linear and organised approach to code reading and understanding. This structured approach may involve carefully analysing the

code line by line, identifying meanings in the code, and logically following the sequence of instructions. By maintaining this approach, HG was less likely to overlook important details such as logic flow and other code intricacies, resulting in better program comprehension. Additionally, HG tended to make use of the data and notepad as valuable tools for information processing during program comprehension. They may find these tools helpful in dry running the code, tracking variables and identifying patterns, allowing for more accurate analysis and interpretation of the code. Our findings elucidate the subtle differences between high- and low-performing students in their approach to navigating through the code for comprehension. These differences may explain why employing the same general walkthrough strategy could lead to varying degrees of success among students, as observed by [Fitzgerald et al. \(2005\)](#).

Retrospective think aloud on video recall was undertaken to collect qualitative data in conjunction with individual scanpaths. The rich qualitative data were coded and analysed to validate the quantitative findings obtained from fixation measurements, visual transitions and trending scanpaths. They were also used to gain insights into participants' use of SRL strategies for program comprehension. The findings suggest that LG and HG differed in terms of their approach to comprehending Program Code 1. LG relied primarily on two specific strategies, namely rehearsal and help-seeking. They repeatedly read the same instructions and sought assistance from notes and hints to compensate for their limited cognitive resources. In contrast, HG demonstrated a more diverse and advanced set of cognitive skills to comprehend the code. They not only jotted down or referred to intermediate steps (elaboration) but also structured their answer in a suitable format when required (organisation). Furthermore, HG demonstrated critical thinking skills by cross-checking the intermediate steps against hints to confirm their answer. When examining Program Code 2, there were significant differences in the strategies employed by LG and HG. LG mainly relied on four key strategies: rehearsal, critical thinking, effort regulation, and help seeking. They attempted to comprehend the code by repeatedly reading the same instructions (rehearsal) and shifting their focus from reading notes to hints (effort regulation) in order to seek direct help from hints (help seeking). They also demonstrated their capacity for critical thinking by skillfully connecting hints with available data to estimate the answer, but this process of estimation may not require the execution of any code. Conversely, HG utilised a more diverse and advanced set of cognitive strategies that could facilitate a deeper understanding of the code. These strategies included elaboration, where they summarised or retrieved key points from the code and notes, jotted down or referred to intermediate steps, and passed data to code for execution. They also used critical thinking to cross-validate the intermediate steps against hints to confirm their answer and verify the answer with the hints. Our findings showcase specific strategies employed by high- and low-performing students. They align with the findings of [McCartney et al. \(2007\)](#) that high-performing students tended to adopt a deep learning approach for constructing a more comprehensive understanding of the code, whereas low-performing students might rely more on surface-level comprehension.

Overall, the findings in this study align with those of [Jessup et al. \(2021\)](#), who argued that skilled programmers tend to focus on intricate code, potentially due to their engagement in a more complicated process of program comprehension. The current study supports this assertion, as HG's approach to code reading involved more frequent utilisation of notepad for higher-order cognitive activities, such as elaboration and critical thinking. Moreover, the results presented in this study are in line with the findings of [Sharif et al.'s \(2012\)](#), which emphasised the importance of devoting sufficient time to scanning the entire code to attain code comprehension. This is corroborated by our finding that HG's code reading approach was more linear and structured, enabling the capture of all details for a more precise interpretation. Our findings also concur with prior research suggesting that less skilled programmers rely more on comments and animations to compensate for their lack of understanding of the code ([Bednarik & Tukiainen, 2006](#); [Crosby & Stelovsky, 1990](#)). This is further supported by our findings that LG accessed supplementary information, such as notes and hints, more frequently to enhance their code reading process. Lastly, the results of this study underscore the importance of employing more advanced cognitive strategies in program comprehension among novice students. This finding further validates [Echeverry et al.'s \(2018\)](#) study, which demonstrated a potential positive correlation between students' utilisation of cognitive strategies and their programming performance.

In response to [RQ1](#), this study demonstrates the feasibility of using a data-driven, systematic approach to understand the use of SRL strategies for program comprehension. This approach involved designing a user interface for program comprehension, collecting eye-tracking data from students (including fixation measurements, visual transitions, trending scanpaths, and video-recalled retrospective think aloud), identifying SRL strategies by associating individual scanpaths with think-aloud data, and quantifying and comparing the use of SRL strategies between different groups of students. This approach provides objective and precise measurements of participants' visual attention and reliable insights into their cognitive processes and strategies. Such insights can be used to corroborate or complement the subjective data collected from self-report questionnaires.

To address [RQ2](#), our findings indicate significant differences between high- and low-performing students in their utilisation of SRL strategies for program comprehension. Low-performing students seemed to rely on rudimentary cognitive and resource management strategy, such as rehearsal and help-seeking, to comprehend program codes. On the other hand, high-performing students deployed more sophisticated cognitive techniques, such as elaboration and critical thinking, to actively execute program codes and assess the accuracy of their comprehension. Furthermore, these findings suggest that the approach and processing of program codes by students could impact their programming performance. This highlights the necessity of cultivating effective strategies for reading and comprehending code among students to enhance their success in programming education.

The implications derived from the findings of this study hold significant potential for shaping an effective pedagogical approach to programming education. From a theoretical standpoint, the observed differences in the utilisation of self-regulated learning (SRL) strategies between high- and low-performing students underscore the pivotal role of advanced cognitive strategies in effective program comprehension. This prompts researchers and practitioners to re-evaluate and potentially refine existing pedagogical models to emphasise the cultivation of these strategies in programming education. In terms of practical application, the findings suggest that instructional interventions should not only deliver programming concepts but also promote the adoption of advanced cognitive techniques, such as elaboration and critical thinking, adapted to diverse programming scenarios for students.

Moreover, the demonstrated effectiveness of a data-driven approach in understanding the application of SRL strategies presents a potent instrument for researchers. By combining objective eye-tracking metrics with retrospective think-aloud insights, a more comprehensive view of students' cognitive processes during program comprehension can be obtained. This blend of quantitative and qualitative data enables educators to pinpoint areas of student proficiency or difficulty, assisting them in offering learning support that resonates with the unique cognitive profiles of each student.

6. Concluding remarks

This study addressed an important research issue regarding the self-regulated learning (SRL) strategies employed by novice students for program comprehension. Traditionally, researchers might approach this issue using self-report questionnaires such as the Motivated Strategies for Learning Questionnaire (Pintrich et al., 1991). However, this study was designed to combine eye-tracking and retrospective think-aloud techniques for more fine-grained and reliable insights into the issue. The findings reveal that low-performing students utilised more basic cognitive and resource management strategies, such as rehearsal and help-seeking. In contrast, high-performing students demonstrated higher-order cognitive skills like elaboration and critical thinking during program comprehension. Consistent with previous studies (Crosby & Stelovsky, 1990; Jessup et al., 2021), these findings provide support for the notion that program comprehension is a cognitively complex process that cannot be solely guided by surface learning approaches, as such approaches are less likely to encourage active engagement with programming activities. Instead, novice students would benefit from employing a diverse and advanced set of SRL strategies to enhance their understanding of program codes.

It is worth noting that the findings of this study may not be generalisable to other contexts due to the limited sample size and the study's focus on program comprehension. In future research, a similar approach could be taken, but with a larger sample size and a focus on different areas, such as program writing and program debugging. Additionally, it would be valuable to explore the potential of using an affordable webcam to monitor and assess students' online programming processes and provide adaptive feedback on their strategy use based on their eye movements. It would also be advantageous to investigate the possible effects of SRL strategy training on students' motivation, engagement, and performance in programming activities.

CRedit authorship contribution statement

Gary Cheng: Conceptualization, Methodology, Formal analysis, Investigation, Writing – original draft, Funding acquisition. **Di Zou:** Investigation, Data curation, Writing – review & editing. **Haoran Xie:** Software, Validation, Formal analysis, Visualization. **Fu Lee Wang:** Data curation, Validation, Visualization.

Data availability

The data that has been used is confidential.

Acknowledgements

Funding: This work was financially supported by the General Research Fund of the Hong Kong Research Grants Council [grant number 18601118]; and The Education University of Hong Kong [grant numbers MIT/DRG05/22-23, RG 6/2022-2034R].

References

- Akyol, G., Sungur, S., & Tekkaya, G. (2010). The contribution of cognitive and metacognitive strategy use to students' science achievement. *Educational Research and Evaluation*, 16(1), 1–21. <https://doi.org/10.1080/13803611003672348>
- Arakawa, K., Hao, Q., Greer, T., Ding, L., Hundhausen, C. D., & Peterson, A. (2021). In *Identification of student self-regulated learning struggles in programming assignments. Proceedings of the 52nd ACM technical symposium on computer science education* (pp. 467–473). New York: ACM. <https://doi.org/10.1145/3408877.3432357>.
- Bednarik, R., & Tukiainen, M. (2006). An eye-tracking methodology for characterizing program comprehension processes. In *ETRA'06: Proceedings of the 2006 Symposium on eye tracking Research and applications* (pp. 125–132). New York: Association for Computing Machinery (ACM). <https://doi.org/10.1145/1117309.1117356>.
- Bednarik, R., & Tukiainen, M. (2008). Temporal eye-tracking data: Evolution of debugging strategies with multiple representations. In *ETRA'08: Proceedings of the 2008 symposium on eye tracking research and applications* (pp. 99–102). New York: Association for Computing Machinery (ACM). <https://doi.org/10.1145/1344471.1344497>.
- Begum, M., Nørberg, J., & Clemmensen, T. (2018). Novice programming strategies. In *Proceedings of the 2018 AIS SIGED international conference on information systems education and research* (Vol. 25). <https://aisel.aisnet.org/siged2018/25>.
- Bennedsen, J., & Caspersen, M. E. (2007). Failure rates in introductory programming – 12 years later. *ACM SIGCSE Bulletin*, 39(2), 32–36.
- Bennedsen, J., & Caspersen, M. E. (2019). Failure rates in introductory programming – 12 years later. *ACM Inroads*, 10(2), 30–36. <https://doi.org/10.1145/3324888>
- Bergin, S., Reilly, R., & Traynor, D. (2005). Examining the role of self-regulated learning on introductory programming performance. In *ICER'05 proceedings of the first annual conference on international computing education research* (pp. 81–86). New York: ACM. <https://doi.org/10.1145/1089786.1089794>.
- Boerman, S. C., & Muller, C. M. (2022). Understanding which cues people use to identify influencer marketing on instagram: An eye tracking study and experiment. *International Journal of Advertising*, 41(1), 6–29. <https://doi.org/10.1080/02650487.2021.1986256>
- Brand, J., Diamond, S. G., Thomas, N., & Gilbert-Diamond, D. (2021). Evaluating the data quality of the Gazepoint GP3 low-cost eye tracker when used independently by study participants. *Behavior Research Methods*, 53, 1502–1514. <https://doi.org/10.3758/s13428-020-01504-2>
- Brooks, R. (1983). Towards a theory of the comprehension of computer programs. *International Journal of Man-Machine Studies*, 18(6), 543–554. [https://doi.org/10.1016/S0020-7373\(83\)80031-5](https://doi.org/10.1016/S0020-7373(83)80031-5)

- Busjahn, T., Bednarik, R., Begel, A., Crosby, M., Paterson, J., Schulte, C., Sharif, B., & Tamm, S. (2015). Eye movements in code reading: Relaxing the linear order. In *ICPC'15: Proceedings of the 2015 IEEE 23rd international conference on program comprehension* (pp. 255–265). IEEE Press.
- Cakiroglu, U. (2014). Analyzing the effect of learning styles and study habits of distance learners on learning performances: A case of an introductory programming course. *International Review of Research in Open and Distance Learning*, 15(4), 161–185. <https://doi.org/10.19173/irrodl.v15i4.1840>
- Cakiroglu, U., & Ozturk, M. (2017). Flipped classroom with problem based activities: Exploring self-regulated learning in a programming language course. *Educational Technology & Society*, 20(1), 337–349.
- Charmaz, K. (2014). *Constructing grounded theory* (2nd ed.). Thousand Oaks, CA: Sage.
- Cheng, G., & Chau, J. (2013). Exploring the relationship between students' self-regulated learning ability and their ePortfolio achievement. *Internet Higher Educ.*, 17, 9–15. <https://doi.org/10.1016/j.iheduc.2012.09.005>
- Cheng, G., Poon, L. K. M., Lau, W. W. F., & Zhou, R. C. (2019). *Exploring the Relationship between Self-Regulated Learning Strategies and Computer Programming Achievement in Higher Education*. In (pp. 67–74). Kuala Lumpur, Malaysia: Proceedings of the 5th International Conference on Education. <https://doi.org/10.17501/24246700.2019.5108>.
- Cohen, J. (1988). *Statistical power analysis for the behavioral sciences* (2nd ed.). Hillsdale, NJ: Lawrence Erlbaum Associates.
- Conley, Q., Earnshaw, Y., & McWatters, G. (2020). Examining course layouts in blackboard: Using eye-tracking to evaluate usability in a learning management system. *International Journal of Human-Computer Interaction*, 36(4), 373–385. <https://doi.org/10.1080/10447318.2019.1644841>
- Crosby, M. E., & Stelovsky, J. (1990). How do we read algorithms? A case study. *Computer*, 23(1), 25–35. <https://doi.org/10.1109/2.48797>
- Cuve, H. C., Stojanov, J., Roberts-Gaal, X., Catmur, C., & Bird, G. (2021). Validation of Gazepoint low-cost eye-tracking and psychophysiology bundle. *Behavior Research Methods*. <https://doi.org/10.3758/s13428-021-01654-x>
- Echeverry, J. J. R., Rosales-Castro, L. F., Restrepo-Calle, F., & González, F. A. (2018). Self-regulated learning in a computer programming course. *IEEE Revista Iberoamericana de Tecnologías del Aprendizaje*, 13(2), 75–83. <https://doi.org/10.1109/RITA.2018.2831758>
- Eraslan, S., Yesilada, Y., & Harper, S. (2016). Scanpath Trend analysis on web pages: Clustering eye tracking scanpaths. *ACM Transactions on the Web*, 10(4). <https://doi.org/10.1145/2970818>. Article 20 (35 pages).
- Falkner, K., Vivian, R., & Falkner, N. J. G. (2014). *Identifying computer science self-regulated learning strategies*. In (pp. 291–296). New York: Association for Computing Machinery (ACM). <https://doi.org/10.1145/2591708.2591715>.
- Feurzeig, W., Papert, S., Bloom, M., Grant, R., & Solomon, C. (1969). Programming-languages as a conceptual framework for teaching Mathematics. In *Final report on the first fifteen months of the Logo project*. Retrieved from <https://eric.ed.gov/?id=ED038034>.
- Fitzgerald, S., Simon, B., & Thomas, L. (2005). Strategies that students use to trace code: An analysis based in grounded theory. In *ICER'05: Proceedings of the first international workshop on computing education research* (pp. 69–80). New York: Association for Computing Machinery (ACM). <https://doi.org/10.1145/1089786.1089793>.
- Greene, J. A., Copeland, D. Z., Deekens, V. M., & Yu, S. B. (2018). Beyond knowledge: Examining digital literacy's role in the acquisition of understanding in science. *Computers & Education*, 117, 141–159. <https://doi.org/10.1016/j.compedu.2017.10.003>
- Greene, J. A., Yu, S. B., & Copeland, D. Z. (2014). Measuring critical components of digital literacy and their relationship with learning. *Computers & Education*, 76, 55–69. <https://doi.org/10.1016/j.compedu.2014.03.008>
- Grover, S., & Pea, R. (2013). Computational thinking in K-12: A review of the state of the field. *Educational Researcher*, 42(1), 38–43. <https://doi.org/10.3102/0013189X12463051>
- Hawi, N. (2010). Causal attributions of success and failure made by undergraduate students in an introductory-level computer programming course. *Computers & Education*, 54(4), 1127–1146. <https://doi.org/10.1016/j.compedu.2009.10.020>
- Hessels, R. S., & Hooge, I. T. C. (2019). Eye tracking in developmental cognitive neuroscience – the good, the bad and the ugly. *Developmental Cognitive Neuroscience*, 40, Article 100710. <https://doi.org/10.1016/j.dcn.2019.100710>
- Holmqvist, K., Nyström, M., Andersson, R., Dewhurst, R., Jarodzka, H., & Van de Weijer, J. (2011). *Eye tracking: A comprehensive guide to methods and measures*. Oxford: OUP.
- Jessup, S. A., Willis, S. M., Alarcon, G. M., & Lee, M. A. (2021). Using eye-tracking data to compare differences in code comprehension and code perceptions between experts and novice programmers. In *Proceedings of the 54th Hawaii international conference on system sciences* (pp. 114–123). University of Hawaii at Manoa.
- Keuning, H., Jeurig, J., & Heeren, B. (2018). A systematic literature review of automated feedback generation for programming exercises. *ACM Transactions on Computing Education*, 19(1). <https://doi.org/10.1145/3231711>. Article 3 (43 pages).
- Koulouri, T., Lauria, S., & Macredie, R. D. (2014). Teaching introductory programming: A quantitative evaluation of different approaches. *ACM Transactions on Computing Education*, 14(4). <https://doi.org/10.1145/2662412>. Article 26 (28 pages).
- Landis, J. R., & Koch, G. G. (1977). The measurement of observer agreement for categorical data. *Biometrics*, 33(1), 159–174. <https://doi.org/10.2307/2529310>
- Lye, S. Y., & Koh, J. H. L. (2014). Review on teaching and learning of computational thinking through programming: What is next for K-12? *Computers in Human Behavior*, 41, 51–61. <https://doi.org/10.1016/j.chb.2014.09.012>
- Malik, S. I., & Coldwell-Neilson, J. (2017). Impact of a new teaching and learning approach in an introductory programming course. *Journal of Educational Computing Research*, 55(6), 789–819. <https://doi.org/10.1177/0735633116685852>
- von Mayrhauser, A., & Vans, A. M. (1995). Program comprehension during software maintenance and evolution. *Computer*, 28(8), 44–55. <https://doi.org/10.1109/2.402076>
- von Mayrhauser, A., Vans, A. M., & Howe, A. E. (1997). Program understanding behaviour during enhancement of large-scale software. *Journal of Software Maintenance: Research and Practice*, 9(5), 299–327. [https://doi.org/10.1002/\(SICI\)1096-908X\(199709/10\)9:5<299::AID-SMR157>3.0.CO;2-S](https://doi.org/10.1002/(SICI)1096-908X(199709/10)9:5<299::AID-SMR157>3.0.CO;2-S)
- McCartney, R., Eckerdal, A., Mostrom, J. E., Sanders, K., & Zander, C. (2007). Successful students' strategies for getting unstuck. *ACM SIGCSE Bulletin*, 39(3), 156–160. <https://doi.org/10.1145/1269900.1268831>
- Medeiros, R. P., Ramalho, G. L., & Falcão, T. P. (2019). A systematic literature review on teaching and learning introductory programming in higher education. *IEEE Transactions on Education*, 62(2), 77–90. <https://doi.org/10.1109/TE.2018.2864133>
- Mehmood, E., Abid, A., Farooq, M. S., & Nawaz, N. A. (2020). Curriculum, teaching and learning, and assessments for introductory programming course. *IEEE Access*, 8, 125961–125981. <https://doi.org/10.1109/ACCESS.2020.3008321>
- Michalsky, T. (2021). Preservice and inservice teachers' noticing of explicit instruction for self-regulated learning strategies. *Frontiers in Psychology*, 12, Article 630197. <https://doi.org/10.3389/fpsyg.2021.630197>
- Obaidallah, U., Haek, M. A., & Cheng, P. C.-H. (2019). A survey on the usage of eye-tracking in computer programming. *ACM Computing Surveys*, 51(1), 1–58. <https://doi.org/10.1145/3145904>
- O'Brien, M. P., Buckley, J., & Shaft, T. M. (2004). Expectation-based, inference-based, and bottom-up software comprehension. *Journal of Software Maintenance and Evolution: Research and Practice*, 16(6), 427–447. <https://doi.org/10.1002/smr.307>
- Papert, S. (1980). *Mindstorms: Children, computers, and powerful ideas*. New York, NY: Basic Books.
- Papert, S. (2000). What's the big idea? Toward a pedagogy of idea power. *IBM Systems Journal*, 39(3/4), 720–729.
- Pedrosa, D., Cravino, J., Morgado, L., & Barreira, C. (2016). Self-regulated learning in computer programming: Strategies students adopted during an assignment. In C. Allison, L. Morgado, J. Pirker, D. Beck, J. Richter, & C. Gütl (Eds.), *Communications in computer and information science: Vol. 621. Immersive learning research network. iLRN 2016* (pp. 87–101). Cham: Springer. https://doi.org/10.1007/978-3-319-41769-1_7
- Pennington, N. (1987). Stimulus structures and mental representations in expert comprehension of computer programs. *Cognitive Psychology*, 19(3), 295–341. [https://doi.org/10.1016/0010-0285\(87\)90007-7](https://doi.org/10.1016/0010-0285(87)90007-7)
- Pereira, F. D., Oliveira, E. H. T., Oliveira, D. B. F., Cristea, A. I., Carvalho, L. S. G., Fonseca, S. C., Toda, A., & Isotani, S. (2020). Using learning analytics in the amazonas: Understanding students' behaviour in introductory programming. *British Journal of Educational Technology*, 51(4), 955–972. <https://doi.org/10.1111/bjet.12953>

- Pintrich, P. R. (1999). The role of motivation in promoting and sustaining self-regulated learning. *International Journal of Educational Research*, 31(6), 459–470. [https://doi.org/10.1016/S0883-0355\(99\)00015-4](https://doi.org/10.1016/S0883-0355(99)00015-4)
- Pintrich, P. R. (2000). Multiple goals, multiple pathways: The role of goal orientation in learning and achievement. *Journal of Educational Psychology*, 92(3), 544–555. <https://doi.org/10.1037/0022-0663.92.3.544>
- Pintrich, P. R. (2004). A conceptual framework for assessing motivation and self-regulated learning in college students. *Educational Psychology Review*, 16(4), 385–407. <https://doi.org/10.1007/s10648-004-0006-x>
- Pintrich, P. R., Smith, D. A. F., García, T., & McKeachie, W. J. (1991). *A manual for the use of the motivated strategies for learning questionnaire (MSLQ)*. Ann Arbor: University of Michigan, National Center for Research to Improve Postsecondary Teaching and Learning.
- Pintrich, P. R., Smith, D. A. F., García, T., & McKeachie, W. J. (1993). Reliability and predictive validity of the motivated strategies for learning questionnaire (MSLQ). *Educational and Psychological Measurement*, 53, 801–813. <https://doi.org/10.1177/0013164493053003024>
- Rahal, R.-M., & Fiedler, S. (2019). Understanding cognitive and affective mechanisms in social psychology through eye-tracking. *Journal of Experimental Social Psychology*, 85, Article 103842. <https://doi.org/10.1016/j.jesp.2019.103842>
- Robillard, M. O., Coelho, W., & Murphy, G. C. (2004). How effective developers investigate source code: An exploratory study. *IEEE Transactions on Software Engineering*, 30(12), 889–903. <https://doi.org/10.1109/TSE.2004.101>
- Romero, M., Lepage, A., & Lille, B. (2017). Computational thinking development through creative programming in higher education. *International Journal of Educational Technology in Higher Education*, 14(42). <https://doi.org/10.1186/s41239-017-0080-z>
- Schulte, C., Clear, T., Taherkhani, A., Busjahn, T., & Paterson, J. H. (2010). An introduction to program comprehension for computer science educators. In *Proceedings of the 2010 ITICSE working group reports* (pp. 65–86). New York: Association for Computing Machinery (ACM). <https://doi.org/10.1145/1971681.1971687>
- Shaft, T. M., & Vessey, I. (1995). Research report – the relevance of application domain knowledge: The case of computer program comprehension. *Information Systems Research*, 6(3), 286–299. <https://doi.org/10.1287/isre.6.3.286>
- Sharif, B., Falcone, M., & Maletic, J. I. (2012). An eye-tracking study on the role of scan time in finding source code defects. In *ETRA'12: Proceedings of symposium on eye tracking research and applications* (pp. 381–384). New York: Association for Computing Machinery (ACM). <https://doi.org/10.1145/2168556.2168642>
- Shaw, R.-S. (2012). A study of the relationships among learning styles, participation types, and performance in programming language learning supported by online forums. *Computers & Education*, 58(1), 111–120. <https://doi.org/10.1016/j.compedu.2011.08.013>
- Shneiderman, B., & Mayer, R. (1979). Syntactic/semantic interactions in programmer behavior: A model and experimental results. *International Journal of Computer & Information Sciences*, 8(3), 219–238. <https://doi.org/10.1007/BF00977789>
- Soloway, E., & Ehrlich, K. (1984). Empirical studies of programming knowledge. *IEEE Transactions on Software Engineering*, 10(5), 595–609. <https://doi.org/10.1109/TSE.1984.5010283>. SE.
- Song, D., Hong, H., & Oh, E. Y. (2021). Applying computational analysis of novice learners' computer programming patterns to reveal self-regulated learning, computational thinking, and learning performance. *Computers in Human Behavior*, 120, Article 106746. <https://doi.org/10.1016/j.chb.2021.106746>
- Tsai, C.-Y. (2019). Improving students' understanding of basic programming concepts through visual programming language: The role of self-efficacy. *Computers in Human Behavior*, 95, 224–232. <https://doi.org/10.1016/j.chb.2018.11.038>
- Uwano, H., Nakamura, M., Monden, A., & Matsumoto, K. (2006). Analyzing individual performance of source code review using reviewers' eye movement. In *ETRA'06: Proceedings of the 2006 symposium on eye tracking research and applications* (pp. 133–140). New York: Association for Computing Machinery (ACM). <https://doi.org/10.1145/1117309.1117357>
- VanderStoep, S. W., & Pintrich, P. R. (2008). *Learning to learn: The skill and will of college success* (2nd ed.). Upper Saddle River, NJ: Prentice Hall.
- Veerasamy, A. K., D'Souza, D., Lindén, R., & Laakso, M.-J. (2019). Relationship between perceived problem-solving skills and academic performance of novice learners in introductory programming courses. *Journal of Computer Assisted Learning*, 35(2), 246–255. <https://doi.org/10.1111/jcal.12326>
- Watson, C., & Li, F. W. B. (2014). Failure rates in introductory programming revisited. In *Proceedings of the 2014 conference on innovation & technology in computer science education* (pp. 39–44). New York: Association for Computing Machinery (ACM). <https://doi.org/10.1145/2591708.2591749>
- Wing, J. M. (2006). Computational thinking. *Communications of the ACM*, 49(3), 33–35. <https://doi.org/10.1145/1118178.1118215>
- Yan, O.-S., & Cheng, G. (2017). Exploring the Impact of Flipped Classroom on Students' Acceptance of Programming in Secondary Education. In *Proceedings of 2017 IEEE 6th International Conference on Teaching, Assessment, and Learning for Engineering* (pp. 246–249). Hong Kong: IEEE. <https://doi.org/10.1109/TALE.2017.8252341>
- Yukselturk, E., & Bulut, S. (2005). Relationships among self-regulated learning components, motivational beliefs and computer programming achievement in an online learning environment. *Mediterranean Journal of Educational Studies*, 10(1), 91–112.
- Zhang, R., Cheng, G., & Chen, X. (2020). Game-based self-regulated language learning: Theoretical analysis and bibliometrics. *PLoS One*, 15(12), Article e0243827. <https://doi.org/10.1371/journal.pone.0243827>
- Zheng, J., Xing, W., Huang, X., Li, S., Chen, G., & Xie, C. (2023). The role of self-regulated learning on science and design knowledge gains in engineering projects. *Interactive Learning Environments*, 31(1), 87–99. <https://doi.org/10.1080/10494820.2020.1761837>
- Zimmerman, B. J. (2000). Attainment of self-regulation: A social cognitive perspective. In M. Boekaerts, P. Pintrich, & M. Zeidner (Eds.), *Self-regulation: Theory, research, and applications* (pp. 13–39). San Diego, CA: Academic Press. <https://doi.org/10.1016/B978-012109890-2/50031-7>